

Monte-Carlo

Calculer ce qu'on ne sait pas calculer,
en lançant les dés

Du jeu de cailloux au moteur de rendu et à l'IA de jeu

Table des matières

0	Avant-propos	1
0.1	À qui s'adresse ce cours	1
0.2	Ce que ce cours contient	1
0.3	Ce que ce cours ne contient pas	1
0.4	Le code de ce cours	2
1	Compter sans compter	3
1.1	La flaque dans la cour	3
1.2	Le premier vrai calcul : trouver π avec de la pluie	3
1.3	Pourquoi ça marche vraiment	4
2	La règle générale	5
2.1	Ce qu'on calcule vraiment : une intégrale	5
2.2	L'estimateur	5
2.3	Pourquoi diviser par p — le point clé	5
2.3.1	Le cas uniforme, que vous connaissez déjà	6
2.4	Non biaisé : viser juste sans être précis	6
3	La précision, et sa loi impitoyable	8
3.1	La loi du $1/\sqrt{N}$	8
3.2	La bonne nouvelle : la dimension ne compte pas	8
3.3	Estimer sa propre erreur	9
4	Viser juste : réduire le bruit sans acheter de machine	11
4.1	L'échantillonnage d'importance	11
4.1.1	Comment tirer selon une densité choisie	11
4.2	La stratification : gratuite, et à faire en premier	12
4.3	La roulette russe : arrêter sans mentir	12
5	Mise en œuvre I : le rendu d'images	14
5.1	Le problème : une équation qu'on ne peut pas résoudre	14
5.2	La transformation en algorithme	14
5.3	Le choix de p , et la simplification qui en découle	15
5.4	Le chemin complet	16
5.5	Pourquoi le rendu direct des lumières change tout	17
6	Mise en œuvre II : jouer sans savoir évaluer	18
6.1	Le problème des méthodes classiques	18
6.2	Le raffinement : ne pas répartir équitablement	18
6.3	D'où vient la valeur d'un nœud	19
6.3.1	Deux nombres, et rien d'autre	19
6.3.2	Pourquoi des parties jouées au hasard informent quand même	19
6.3.3	Un exemple chiffré, à faire à la main	20
6.3.4	On ne compare jamais que des frères	20
6.3.5	Est-ce que ça converge vraiment?	21
6.4	Les quatre phases, répétées en boucle	21
6.5	Le code complet	21
6.6	Ce que MCTS vous donne gratuitement	23
7	Les pièges	24
7.1	Le générateur aléatoire	24
7.2	Le biais silencieux	25
7.3	Mécanisme de la simulation	25

Avant-propos

Il existe des calculs qu'on ne sait pas faire. Pas « difficiles » : *pas faisables*. La quantité de lumière qui arrive sur un pixel, la valeur d'un coup au milieu d'une partie, la surface d'une forme qui n'a pas d'équation — ce sont des sommes infinies sur des espaces à dix, vingt, cent dimensions. Aucune formule ne les résout, et aucune machine n'a assez de temps pour les parcourir.

Monte-Carlo est la réponse à cette impasse, et c'est une réponse presque insolente : **si on ne peut pas tout regarder, regardons au hasard, et comptons**. Le résultat n'est jamais exact. Il est *presque* exact, et il s'améliore tant qu'on lui donne du temps.

0.1 À qui s'adresse ce cours

À quelqu'un qui sait programmer, et qui n'a pas besoin d'être à l'aise en mathématiques pour commencer. Le chapitre 1 se lit sans une seule formule; les formules n'arrivent qu'au chapitre 2, une fois que l'intuition est en place, et chacune est expliquée en français avant d'être écrite en symboles.

0.2 Ce que ce cours contient

- l'idée, avec des cailloux et une flaque d'eau (chapitre 1);
- la règle générale, et le seul point que tout le monde rate — la division par la densité de tirage (chapitre 2);
- pourquoi la précision progresse si lentement, et pourquoi c'est quand même la meilleure méthode du monde en grande dimension (chapitre 3);
- comment réduire le bruit sans acheter une machine plus puissante (chapitre 4);
- deux mises en œuvre complètes, pas à pas, avec le code : le rendu d'images (chapitre 5) et l'intelligence artificielle de jeu de plateau (chapitre 6);
- les pièges qui coûtent une journée, dont trois qui ne préviennent pas (chapitre 7).

0.3 Ce que ce cours ne contient pas

Pas de démonstration de la loi des grands nombres ni du théorème central limite : ils sont *utilisés*, énoncés, et leur conséquence pratique est expliquée — mais les démontrer demanderait un cours de probabilités, et ne rendrait personne meilleur programmeur.

Pas non plus de chaînes de Markov (Metropolis-Hastings, MCMC). C'est la suite naturelle de ce cours, et elle mérite le sien.

Le contrat de Monte-Carlo, en une phrase

On échange l'**exactitude** contre la **possibilité de calculer**. Une méthode exacte qui ne termine jamais ne vaut rien; une méthode approchée qui rend une réponse utilisable en dix secondes, et une réponse excellente en dix minutes, vaut tout.

0.4 Le code de ce cours

Les extraits suivent les conventions du moteur Nkentseu : types en `NkPascalCase`, méthodes en `PascalCase`, membres privés en `mCamelCase`, types primitifs `float32` / `nk_uint32`, et aucune dépendance à la bibliothèque standard. Ils sont écrits pour être lus, puis copiés : chacun tourne tel quel.

Compter sans compter

Ce chapitre ne contient aucune formule. Il contient l'idée entière.

1.1 La flaque dans la cour

Imaginez une flaque d'eau de forme bizarre au milieu d'une cour rectangulaire. Vous voulez savoir quelle **surface** elle occupe. Vous n'avez pas de règle, et la flaque ne ressemble à aucune figure connue : ce n'est ni un carré, ni un cercle, ni rien qui ait une formule.

Alors vous faites ceci. Vous prenez une poignée de cailloux et vous les lancez **au hasard, partout dans la cour**, sans viser. Puis vous comptez ceux qui ont fait « plouf ».

Le raisonnement complet

Si 30 cailloux sur 100 sont tombés dans la flaque, alors la flaque occupe environ **30 % de la cour**. Et la cour, elle, est un rectangle : sa surface, vous la connaissez.

$$\text{surface de la flaque} \approx 0,30 \times \text{surface de la cour}$$

Vous venez de mesurer l'immesurable sans jamais approcher la flaque. C'est *tout* Monte-Carlo. Le nom vient du casino de Monte-Carlo : on joue aux dés pour obtenir un résultat sérieux.

Deux choses sont déjà vraies, et elles ne changeront plus de tout le cours :

- avec 10 cailloux, la réponse est mauvaise ; avec 10 000, elle est bonne. **Plus on lance, plus c'est juste** ;
- on n'aura **jamais** la réponse exacte. On aura une réponse presque juste, dont l'erreur rétrécit tant qu'on continue.

1.2 Le premier vrai calcul : trouver π avec de la pluie

Passons à un cas où la réponse est connue, pour vérifier que la méthode ne ment pas.

Prenez un carré de côté 2, allant de -1 à $+1$ dans les deux directions, et le cercle de rayon 1 qui tient exactement dedans.

- surface du carré $= 2 \times 2 = 4$;
- surface du cercle $= \pi \times r^2 = \pi$ (puisque $r = 1$).

Un point lancé au hasard dans le carré a donc une chance sur $4/\pi$ de tomber dans le cercle — autrement dit, la proportion de points dans le cercle vaut $\pi/4$. Retournez la relation :

$$\pi \approx 4 \times \frac{\text{points tombés dans le cercle}}{\text{points lancés}}$$

Reste à savoir si un point (x, y) est dans le cercle. Il l'est si sa distance au centre ne dépasse pas 1, c'est-à-dire si $x^2 + y^2 \leq 1$. On ne prend même pas la racine carrée : comparer les carrés donne le même verdict et coûte moins cher.

Estimer π — le programme complet

```
1 float32 EstimerPi(nk_uint32 n) {
2     nk_uint32 dedans = 0;
3     for (nk_uint32 i = 0; i < n; ++i) {
4         const float32 x = math::NkRand.NextFloat32(-1.f, 1.f);
5         const float32 y = math::NkRand.NextFloat32(-1.f, 1.f);
6         // Pas de racine carree : comparer Les carres donne Le meme verdict.
7         if (x * x + y * y <= 1.f)
8             ++dedans;
9     }
10    return 4.f * (float32)dedans / (float32)n;
11 }
```

Voici ce que rend ce programme, en pratique :

Tirages	Estimation	Erreur
100	3,08	2 %
10 000	3,132	0,3 %
1 000 000	3,1416	0,02 %

Ce que vous venez de faire

Vous avez calculé π **sans jamais écrire π** , et sans connaître la moindre formule de géométrie du cercle. Vous n'avez utilisé qu'une chose : la capacité à répondre à la question « ce point est-il dedans ? ».

C'est le motif universel. Monte-Carlo ne demande jamais de savoir résoudre le problème — seulement de savoir **évaluer une réponse ponctuelle**.

1.3 Pourquoi ça marche vraiment

Ce n'est pas une coïncidence ni un bricolage. C'est la *loi des grands nombres* : quand on répète une expérience au hasard un grand nombre de fois, la moyenne des résultats se rapproche de la valeur vraie.

Dit autrement : chaque caillou pris isolément ne vous apprend presque rien (il est dedans ou dehors, c'est tout). Mais leur **moyenne** sait quelque chose qu'aucun caillou ne sait.

Ce n'est pas « ça finit par tomber juste par chance »

Une confusion fréquente : croire que le résultat oscille et qu'on attrape la bonne valeur au passage. Non — la moyenne **converge**. Chaque nouveau tirage tire l'estimation vers la vraie valeur, et l'écart rétrécit de façon prévisible (chapitre 3). Ce n'est pas de la chance, c'est une garantie mathématique.

Vérifier la convergence de ses propres yeux

Modifiez `EstimerPi` pour qu'elle affiche l'estimation courante tous les 1000 tirages, sur 100 000 tirages. Tracez la courbe.

Vous verrez trois choses : elle démarre n'importe où, elle se resserre vite au début, puis elle rampe. Ce ralentissement est le sujet du chapitre 3, et c'est le prix à payer de la méthode.

La règle générale

Le chapitre précédent comptait des cailloux. Celui-ci écrit la même chose en symboles, et ajoute la seule subtilité de toute la méthode.

2.1 Ce qu'on calcule vraiment : une intégrale

Une surface, une quantité de lumière, une moyenne sur toutes les parties possibles : ce sont toutes des **intégrales**. Une intégrale, c'est une somme d'une infinité de valeurs sur un domaine :

$$I = \int_D f(x) dx$$

Lisez-la ainsi : « pour chaque point x du domaine D , prends la valeur $f(x)$, et fais-en la somme ». Le problème est qu'il y a une infinité de points — on ne peut pas les visiter.

2.2 L'estimateur

Monte-Carlo remplace cette somme infinie par une moyenne finie :

La formule centrale, à retenir par cœur

$$\langle I \rangle_N = \frac{1}{N} \sum_{i=1}^N \frac{f(x_i)}{p(x_i)}$$

où les x_i sont N points tirés au hasard selon une densité de probabilité p . Ce nombre s'approche de I quand N grandit.

Le $f(x_i)$ ne pose aucune difficulté : c'est la valeur de ce qu'on somme, au point tiré. Le $\frac{1}{N} \sum$ non plus : c'est une moyenne. Reste le $p(x_i)$ au dénominateur, et c'est lui que tout le monde rate.

2.3 Pourquoi diviser par p — le point clé

Imaginez que vous comptiez les arbres d'une forêt en vous y promenant, puis que vous multipliez par la surface. Si votre promenade passe deux fois plus souvent près de la rivière que sur la colline, vous compterez les arbres de la rivière deux fois trop. Votre estimation sera fautive — non parce que vous avez mal compté, mais parce que **votre façon de vous promener favorisait un endroit**.

La correction est évidente une fois dite : chaque arbre vu doit être divisé par la fréquence avec laquelle vous passiez là. Un arbre vu dans une zone sur-visitée compte moins ; un arbre vu dans une zone rarement visitée compte davantage.

Le rôle de p

La division par p **annule le favoritisme de votre tirage**. Elle transforme n'importe quelle façon de tirer — même très biaisée — en une estimation correcte.

C'est ce qui rend la méthode libre : vous pouvez tirer *où vous voulez*, du moment que vous savez avec quelle probabilité vous y tirez.

2.3.1 Le cas uniforme, que vous connaissez déjà

Si vous tirez uniformément sur un domaine de volume V , alors la densité vaut $p(x) = 1/V$ partout — la même chance partout. Remplacez dans la formule :

$$\langle I \rangle_N = \frac{1}{N} \sum_{i=1}^N \frac{f(x_i)}{1/V} = \frac{V}{N} \sum_{i=1}^N f(x_i)$$

C'est-à-dire : **le volume, multiplié par la moyenne des valeurs**. C'est exactement le calcul de la flaque (surface de la cour $\times$ proportion de plouf) et celui de π (surface du carré $\times$ proportion dans le cercle). Vous utilisiez déjà la formule générale sans le savoir.

2.4 Non biaisé : viser juste sans être précis

Une propriété mérite un mot, parce qu'elle est contre-intuitive.

L'estimateur est dit **non biaisé** : son espérance est exactement I , *dès le premier échantillon*. Avec un seul caillou, votre estimation de la flaque sera ridicule (0 % ou 100 %) — mais elle n'est pas *décalée* : si vous refaisiez l'expérience à un caillou des millions de fois, la moyenne de toutes ces estimations ridicules donnerait la bonne surface.

Justesse et précision ne sont pas la même chose

- **Non biaisé** = qui vise le centre de la cible. Les impacts sont dispersés, mais leur centre est le bon.
- **Précis** = qui groupe ses impacts. Ils peuvent être groupés à côté de la cible.

Monte-Carlo est *juste* d'emblée et devient *précis* avec le temps. Un estimateur biaisé, lui, est stable et faux : il ne bruite pas, il ment. Et c'est bien plus grave, parce que rien ne le signale.

Deux conditions à ne jamais oublier

La formule n'est valable que si :

1. $p(x) > 0$ **partout où** $f(x) \neq 0$. Si vous ne tirez jamais dans une région où la fonction vaut quelque chose, cette contribution est perdue pour toujours — et aucun nombre d'échantillons ne la rattrapera. En rendu, c'est la lumière qu'on ne verra jamais parce qu'aucun rayon ne va dans sa direction ;
2. p est une **vraie** densité : positive, et d'intégrale 1. Une densité mal normalisée décale silencieusement tout le résultat.

Vérifier que la division par p est indispensable

Estimez $\int_0^1 x^2 dx$ (la réponse exacte est $1/3$) de deux façons :

1. en tirant x uniformément dans $[0, 1]$, donc $p(x) = 1$;

2. en tirant $x = \sqrt{u}$ avec u uniforme — ce qui favorise les grandes valeurs de x . La densité correspondante est $p(x) = 2x$.

Faites le second calcul **en oubliant volontairement** de diviser par $2x$: vous trouverez $1/2$ au lieu de $1/3$. Puis divisez : vous retrouvez $1/3$. L'erreur de 50 % ne vient pas du hasard — elle vient du favoritisme non corrigé.

La précision, et sa loi impitoyable

Ce chapitre contient la mauvaise nouvelle de la méthode, puis la raison pour laquelle on l'emploie quand même — et cette raison est écrasante.

3.1 La loi du $1/\sqrt{N}$

On aimerait que doubler le nombre d'échantillons divise l'erreur par deux. Ce n'est pas le cas. L'erreur type de l'estimateur vaut :

$$\sigma_N = \frac{\sigma}{\sqrt{N}}$$

où σ est l'écart-type de la fonction échantillonnée. La décroissance se fait en **racine carrée** du nombre d'échantillons.

Le chiffre à retenir, celui qui explique tout

Pour diviser le bruit par 2, il faut 4 fois plus d'échantillons. Pour le diviser par 10, il en faut 100 fois plus.

C'est exactement pourquoi une image en *path tracing* met des heures à devenir propre : passer d'une image granuleuse à une image lisse ne coûte pas un peu plus cher, il coûte cent fois plus cher.

Échantillons	Bruit relatif	Temps (base 1)
100	10 %	1
400	5 %	4
10 000	1 %	100
1 000 000	0,1 %	10 000

Deux conséquences pratiques, immédiates :

- **inutile d'espérer la perfection par la force.** Multiplier la machine par deux ne réduit le bruit que de 30 %. Le levier n'est pas la puissance, c'est la réduction de la variance σ (chapitre 4) ;
- **le rendement décroît.** Les premières secondes d'un rendu apportent l'essentiel ; les dernières heures polissent. Un moteur qui affiche l'image *pendant* qu'elle converge est donc utilisable bien avant d'être terminé — c'est le principe du rendu progressif.

3.2 La bonne nouvelle : la dimension ne compte pas

Voici pourquoi Monte-Carlo est irremplaçable, et non pas seulement commode.

Les méthodes classiques d'intégration (rectangles, trapèzes, quadrature de Gauss) découpent le domaine en une grille. En dimension d , une grille de n points par axe demande n^d points au total. Regardez la catastrophe :

Dimension	Points d'une grille 10×	Faisable?
1	10	oui
2	100	oui
5	100 000	oui
10	10 000 000 000	non
20	10 ²⁰	jamais

C'est ce qu'on appelle la *malédiction de la dimension*. Et voici le point décisif :

Le $1/\sqrt{N}$ ne dépend pas de la dimension

L'erreur de Monte-Carlo vaut $\sigma/\sqrt{N}$ **en dimension 1 comme en dimension 100**. La dimension n'apparaît nulle part dans la formule.

En rendu, chaque rebond de lumière ajoute deux dimensions (la direction du rebond). Un chemin à cinq rebonds est une intégrale en dimension 10, et personne ne le remarque : le code est le même, le coût est le même. Aucune autre méthode ne tient à ce régime.

Autrement dit, on ne choisit pas Monte-Carlo parce qu'il est bon — il est lent. On le choisit parce qu'au-delà de quelques dimensions, **il est le seul qui existe encore**.

3.3 Estimer sa propre erreur

Un estimateur honnête sait dire à quel point il est sûr de lui. Comme l'écart-type σ est inconnu, on l'estime à partir des mêmes échantillons :

$$s^2 = \frac{1}{N-1} \sum_{i=1}^N (y_i - \bar{y})^2 \quad \text{avec} \quad y_i = \frac{f(x_i)}{p(x_i)}$$

et l'erreur type de la moyenne vaut alors $s/\sqrt{N}$.

Accumulateur qui rend la moyenne ET sa barre d'erreur

```

1 // Algorithme de Welford : une seule passe, et surtout STABLE. La formule
2 // naïve (somme des carrés moins carré de la somme) soustrait deux grands
3 // nombres presque égaux et perd toute la précision -- elle rend même des
4 // variances NEGATIVES sur de longues séries.
5 class NkAccumulateurMC {
6     public:
7         void Ajouter(float64 y) {
8             mN += 1;
9             const float64 delta = y - mMoyenne;
10            mMoyenne += delta / (float64)mN;
11            mM2 += delta * (y - mMoyenne); // Le second delta est le NOUVEAU
12        }
13
14        float64 Moyenne() const { return mMoyenne; }
15
16        // Erreur type de la moyenne : l'intervalle « moyenne +/- 2 sigma »
17        // contient la vraie valeur environ 95 fois sur 100.
18        float64 ErreurType() const {
19            if (mN < 2)
20                return 0.0;
21            const float64 variance = mM2 / (float64)(mN - 1);
22            return NkMath::Sqrt(variance / (float64)mN);

```

```

23     }
24
25     private:
26         nk_uint64 mN = 0;
27         float64 mMoyenne = 0.0;
28         float64 mM2 = 0.0;
29 };

```

À quoi sert la barre d'erreur, concrètement

Elle permet d'**arrêter au bon moment**. Plutôt que de fixer un nombre d'échantillons au hasard, on continue jusqu'à ce que l'erreur type passe sous un seuil. Un moteur de rendu peut ainsi donner plus d'échantillons aux pixels bruyants (les zones d'ombre douce, les caustiques) et moins aux pixels déjà propres — à budget égal, l'image entière converge plus vite.

La barre d'erreur ne voit pas ce qu'elle n'a pas tiré

Si une contribution énorme se cache dans une région minuscule que vos échantillons n'ont jamais visitée, votre estimateur sera *confiant et faux* : peu de variance observée, donc petite barre d'erreur, et pourtant à côté de la plaque. C'est le cas classique d'une petite source de lumière très intense. La barre d'erreur mesure la dispersion de ce que vous avez vu — jamais l'existence de ce que vous n'avez pas vu.

Constater la loi de la racine

Estimez π 50 fois avec $N = 100$, puis 50 fois avec $N = 400$, puis 50 fois avec $N = 1600$. Calculez à chaque fois l'écart-type de vos 50 estimations. Vous devez trouver un rapport d'environ 2 entre chaque palier — pas 4. Si vous trouvez 4, votre générateur aléatoire produit des tirages corrélés (chapitre 7).

Viser juste : réduire le bruit sans acheter de machine

On ne peut pas battre le $1/\sqrt{N}$: c'est une loi. Mais on peut réduire le σ qui le multiplie — et là, les gains ne se comptent pas en pourcents, ils se comptent en facteurs.

4.1 L'échantillonnage d'importance

L'idée tient en une image. Pour trouver les poissons d'un lac, ne quadrillez pas le lac uniformément : jetez votre filet là où l'eau bouge.

Formellement : choisissez une densité p qui **ressemble à f** . Plus p est grande là où f est grande, plus le rapport f/p est régulier — et c'est la régularité de ce rapport qui fait la faible variance.

Le cas idéal, qui explique tout le reste

Si $p(x) = f(x)/I$ exactement, alors chaque terme de la somme vaut

$$\frac{f(x_i)}{p(x_i)} = \frac{f(x_i)}{f(x_i)/I} = I$$

c'est-à-dire la réponse, à **chaque tirage**. La variance tombe à zéro : un seul échantillon suffirait.

C'est inatteignable — il faudrait connaître I , qui est justement ce qu'on cherche. Mais ça donne la direction : **plus p imite f , moins ça bruite**. Toute la pratique de Monte-Carlo consiste à fabriquer des p approchantes et bon marché.

4.1.1 Comment tirer selon une densité choisie

Reste une question concrète : votre générateur ne sait produire que de l'uniforme entre 0 et 1. Comment en tirer un nombre distribué selon p ?

Par la **méthode de l'inverse**. Calculez la fonction de répartition $P(x) = \int_{-\infty}^x p(t) dt$, inversez-la, et appliquez cette inverse à un tirage uniforme u .

Exemple complet : tirer selon $p(x) = 2x$ sur $[0, 1]$

```
1 // 1. Repartition : P(x) = integrale de 0 a x de 2t dt = x^2
2 // 2. Inversion : u = x^2 => x = sqrt(u)
3 // 3. Tirage :
4 float32 TirerSelonDeuxX() {
5     const float32 u = math::NkRand.NextFloat32(); // uniforme [0,1)
6     return NkMath::Sqrt(u);
7 }
8 // Et la densité à rendre à l'appelant, pour la division :
9 float32 DensiteDeuxX(float32 x) { return 2.f * x; }
```

Quand l'inverse n'est pas calculable, on emploie le *rejet* : tirer dans une enveloppe simple et jeter ce qui dépasse. C'est plus coûteux, mais toujours correct.

4.2 La stratification : gratuite, et à faire en premier

Cent points tirés purement au hasard laissent des trous et forment des grappes. Le hasard est *grumeleux*, contrairement à ce que l'intuition suggère.

Découpez le domaine en cases et tirez un point **dans chaque case** : même nombre d'échantillons, même coût, mais couverture régulière.

Stratification en une dimension

```
1 // Sans stratification : n tirages libres, avec trous et grappes.
2 for (nk_uint32 i = 0; i < n; ++i)
3     total += f(math::NkRand.NextFloat32());
4
5 // Avec stratification : une case par échantillon, un tirage DANS la case.
6 // Le jitter (le hasard à l'intérieur de la case) est indispensable : sans
7 // lui, on retombe sur une grille régulière, qui réintroduit des motifs
8 // d'aliasing -- des moires visibles à l'écran plutôt qu'un grain neutre.
9 for (nk_uint32 i = 0; i < n; ++i) {
10     const float32 u = ((float32)i + math::NkRand.NextFloat32()) / (float32)n;
11     total += f(u);
12 }
```

La stratification ne passe pas à l'échelle en dimension

En dimension d , stratifier chaque axe en k cases demande k^d échantillons : c'est la malédiction de la dimension qui revient par la fenêtre.

La parade porte un nom : les **échantillons de faible discordance** (Halton, Sobol) et le *Latin hypercube*. Ils gardent la régularité de la stratification sans exiger k^d points. C'est ce qu'utilisent tous les moteurs de rendu modernes, et c'est le prolongement naturel de ce chapitre.

4.3 La roulette russe : arrêter sans mentir

Un chemin de lumière peut rebondir indéfiniment. On doit bien s'arrêter — mais **couper net à une profondeur fixe est un biais** : toute la lumière des rebonds suivants disparaît, et l'image s'assombrit systématiquement.

La roulette russe résout ça élégamment. À chaque rebond, on continue avec une probabilité q , et on **divise la contribution par q** .

Pourquoi c'est exact

Le chemin survit avec la probabilité q et rapporte alors C/q ; il meurt avec la probabilité $1 - q$ et rapporte 0. Son espérance vaut donc

$$q \times \frac{C}{q} + (1 - q) \times 0 = C$$

soit exactement ce qu'il aurait rapporté sans interruption. **On termine plus tôt sans introduire la moindre erreur systématique** — on ajoute seulement un peu de variance.

En pratique, on choisit q proportionnel à l'énergie restante du chemin : un rayon très atténué a peu de chances de survivre, ce qui est exactement ce qu'on veut.

Mesurer le gain de l'échantillonnage d'importance

Estimez $\int_0^1 x^2 dx$ de deux façons, avec le même nombre d'échantillons : uniformément ($p = 1$), puis avec $p(x) = 2x$ (donc $x = \sqrt{u}$, sans oublier de diviser par $2x$).

Lancez chaque version 100 fois et comparez l'écart-type des résultats. La seconde doit bruiser nettement moins — parce que $2x$ ressemble davantage à x^2 que la fonction constante 1. Vous venez de gagner du temps de calcul sans toucher au matériel.

Mise en œuvre I : le rendu d'images

Première application complète, pas à pas. C'est ici que Monte-Carlo n'est pas une astuce mais une nécessité absolue.

5.1 Le problème : une équation qu'on ne peut pas résoudre

La lumière qui repart d'un point x vers l'œil, dans la direction ω_o , vaut ce que la surface émet elle-même plus tout ce qu'elle reçoit et renvoie :

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_{\Omega} L_i(x, \omega_i) f_r(x, \omega_i, \omega_o) \cos \theta_i d\omega_i$$

En français, terme par terme :

- L_e : ce que la surface **émet** (nul, sauf si c'est une lampe);
- $\int_{\Omega}$: la somme sur **toutes les directions** de l'hémisphère au-dessus du point — une infinité de directions;
- $L_i(x, \omega_i)$: la lumière qui **arrive** de la direction ω_i ;
- f_r : la **BRDF**, qui dit quelle fraction de ce qui arrive d'une direction repart vers une autre — la matière;
- $\cos \theta_i$: la lumière rasante étale son énergie sur une plus grande surface, donc elle compte moins. C'est le même effet qui fait qu'il fait plus froid en hiver.

Deux raisons rendent cette équation insoluble : elle est **récursive** (L_i est le L_o d'un autre point, qui dépend d'un autre encore), et son domaine est **continu et infini**.

5.2 La transformation en algorithme

Appliquons la formule du chapitre 2 : au lieu de sommer sur toutes les directions, tirons-en quelques-unes au hasard.

Les six pas, dans l'ordre

1. **Tirer** une direction ω_i selon une densité p choisie par nous;
2. **lancer** un rayon dans cette direction et trouver ce qu'il touche;
3. **recupérer** la lumière qui vient de là — par récursion;
4. **pondérer** : multiplier par $f_r \cdot \cos \theta_i$, puis **diviser** par $p(\omega_i)$;
5. **accumuler** dans la somme du pixel;
6. **recommencer** N fois, puis diviser par N .

5.3 Le choix de p , et la simplification qui en découle

Tirer uniformément sur l'hémisphère fonctionne, mais bruite : on gaspille des rayons dans les directions rasantes, que le $\cos \theta_i$ va presque annuler.

L'échantillonnage en cosinus tire donc plus souvent près de la normale, avec la densité

$$p(\omega_i) = \frac{\cos \theta_i}{\pi}$$

Regardez ce qui se passe quand on remplace, pour une surface diffuse dont la BRDF vaut $f_r = \rho/\pi$:

$$\frac{f_r \cdot \cos \theta_i}{p(\omega_i)} = \frac{\frac{\rho}{\pi} \cos \theta_i}{\frac{\cos \theta_i}{\pi}} = \rho$$

Pourquoi tous les moteurs font ça

Le $\cos \theta_i$ se simplifie avec celui de la densité, et le π aussi. Il ne reste que l'albédo. On a donc, en même temps :

- moins de bruit (on tire là où ça compte);
- moins de calcul (deux termes disparaissent algébriquement).

C'est le prototype du bon échantillonnage d'importance : celui qui rend le code *plus simple* et non plus compliqué.

Tirage en cosinus sur l'hémisphère (methode de Malley)

```
1 // On tire un point uniformément dans le DISQUE unite, puis on le relève sur
2 // l'hémisphère. C'est exactement équivalent à un tirage en cosinus, et ça ne
3 // coûte qu'une racine carrée -- résultat classique, et bien plus rapide que
4 // d'inverser la répartition en angles sphériques.
5 NkVec3 EchantillonnerCosinus(const NkVec3& normale, float32* pdfOut) {
6     const float32 u1 = math::NkRand.NextFloat32();
7     const float32 u2 = math::NkRand.NextFloat32();
8
9     const float32 r = NkMath::Sqrt(u1);
10    const float32 phi = 2.f * NkMath::kPi * u2;
11
12    const float32 x = r * NkMath::Cos(phi);
13    const float32 y = r * NkMath::Sin(phi);
14    const float32 z = NkMath::Sqrt(NkMath::Max(0.f, 1.f - u1)); // = cos(theta)
15
16    // Base orthonormée autour de la normale (methode de Duff : sans branche
17    // conditionnelle, et numériquement stable même quand n.z vaut -1).
18    const float32 sign = (normale.z >= 0.f) ? 1.f : -1.f;
19    const float32 a = -1.f / (sign + normale.z);
20    const float32 b = normale.x * normale.y * a;
21    const NkVec3 t(1.f + sign * normale.x * normale.x * a, sign * b, -sign * normale.x);
22    const NkVec3 s(b, sign + normale.y * normale.y * a, -normale.y);
23
24    if (pdfOut)
25        *pdfOut = z / NkMath::kPi; // p = cos(theta) / pi
26
27    return t * x + s * y + normale * z;
28 }
```

5.4 Le chemin complet

Contribution d'UN chemin – la moyenne de N appels donne le pixel

```
1 NkVec3 TracerChemin(const NkRay& rayon, nk_uint32 profondeur) {
2     NkHit hit;
3     if (!mScene.Intersect(rayon, hit))
4         return CielVersLumiere(rayon.direction); // Le rayon part a l'infini
5
6     NkVec3 radiance = hit.material.emission; // La surface brille-t-elle ?
7
8     // ROULETTE RUSSE. Couper net a une profondeur fixe perdrait toute la
9     // lumiere des rebonds suivants : L'image s'assombrirait de facon
10    // systematique (un BIAIS, pas du bruit). Ici on continue avec la
11    // probabilite q et on divise par q : L'esperance reste exacte.
12    float32 q = 1.f;
13    if (profondeur > 3) {
14        q = 0.8f;
15        if (math::NkRand.NextFloat32() > q)
16            return radiance; // Le chemin meurt ici
17    }
18
19    // (1) tirer une direction, et connaitre SA densite
20    float32 pdf = 0.f;
21    const NkVec3 wi = EchantillonnerCosinus(hit.normal, &pdf);
22    if (pdf <= 0.f)
23        return radiance; // garde : jamais diviser par 0
24
25    // (2) et (3) : suivre le rayon, recuperer ce qui vient de la
26    const NkVec3 entrant = TracerChemin(NkRay(hit.position, wi), profondeur + 1);
27
28    // (4) ponderer : BRDF * cos, DIVISE par la densite et par q.
29    // Avec le tirage en cosinus, cos et pi se simplifient : il ne reste que
30    // l'albedo. On ecrit quand meme la forme complete -- elle reste juste si
31    // l'on change de densite un jour, et c'est la que se cachent les bugs.
32    const float32 cosTheta = NkMath::Max(0.f, NkVec3::Dot(hit.normal, wi));
33    const NkVec3 brdf = hit.material.albedo * (1.f / NkMath::kPi);
34
35    radiance += entrant * brdf * cosTheta / (pdf * q);
36    return radiance;
37 }
38
39 // (5) et (6) : La moyenne sur le pixel.
40 NkVec3 RendrePixel(nk_uint32 px, nk_uint32 py, nk_uint32 echantillons) {
41     NkVec3 somme(0.f, 0.f, 0.f);
42     for (nk_uint32 s = 0; s < echantillons; ++s) {
43         // Jitter dans le pixel : sans lui, tous les rayons passent par le
44         // meme point et les bords sont crenele. Avec, l'anticrenelage est
45         // GRATUIT -- il vient du meme hasard qui sert a l'eclairage.
46         const float32 jx = math::NkRand.NextFloat32();
47         const float32 jy = math::NkRand.NextFloat32();
48         somme += TracerChemin(mCamera.RayonVers((float32)px + jx, (float32)py + jy), 0);
49     }
50     return somme / (float32)echantillons;
51 }
```

Trois erreurs qui donnent une image plausible mais fausse

1. **oublier de diviser par la densité** : l'image reste jolie, mais l'éclairage est faux. Rien ne le signale — c'est le bug n°1 du path tracing;

2. **couper les chemins à profondeur fixe** : image trop sombre, d'autant plus que la scène a d'inter-réflexions;
3. **oublier le jitter dans le pixel** : bords crénelés, qu'aucun nombre d'échantillons ne corrigera jamais — tous les rayons passent exactement au même endroit.

5.5 Pourquoi le rendu direct des lumières change tout

Avec le code ci-dessus, une petite lampe intense est un cauchemar : les rayons tirés au hasard la manquent presque toujours, et les rares qui la touchent rapportent une énergie énorme. Résultat : des **pixels blancs isolés** sur une image sombre, ce qu'on appelle des *fireflies*.

La parade s'appelle *next event estimation* : à chaque rebond, on tire **en plus** un point directement sur une source de lumière, on vérifie qu'il est visible, et on ajoute sa contribution. On combine ensuite les deux estimations (celle par la BRDF et celle par la lumière) avec des poids — c'est l'*échantillonnage à importances multiples*, la brique suivante.

Voir le bruit diminuer

Rendez une scène simple (un plan, une sphère, un ciel uniforme) à 1, 4, 16, 64 puis 256 échantillons par pixel, en gardant les images.

Vérifiez la loi du chapitre 3 : entre 16 et 64 échantillons — quatre fois plus — le grain doit diminuer d'un facteur 2 environ, pas 4. Puis ajoutez le tirage direct des lumières et comparez à nombre d'échantillons égal.

Mise en œuvre II : jouer sans savoir évaluer

Deuxième application complète, dans un tout autre monde : l'intelligence artificielle d'un jeu de plateau. Le principe est identique, les gains sont spectaculaires, et la mise en œuvre tient en une page.

6.1 Le problème des méthodes classiques

Un moteur de jeu traditionnel (Minimax, Negamax, alpha-bêta) explore l'arbre des coups possibles et, à une certaine profondeur, appelle une **fonction d'évaluation** : «cette position vaut +3 pour les blancs».

Cette fonction est le talon d'Achille. Aux échecs, on sait la fabriquer (matériel, structure de pions, sécurité du roi — des siècles de théorie). Mais sur un jeu nouveau, sur le Go, ou sur votre propre jeu de plateau, **personne ne sait dire si une position à mi-partie est bonne**. Et une mauvaise fonction d'évaluation produit une IA confiante et stupide.

L'idée qui contourne le problème

Ne jugeons pas la position. **Jouons la partie jusqu'au bout**, au hasard, et regardons qui gagne. À la fin d'une partie, il n'y a plus rien à évaluer : le résultat est un fait.
«Je ne sais pas si ce coup est bon. Alors depuis ce coup, je joue mille parties complètement au hasard jusqu'à la fin. Si j'en gagne sept cents, ce coup est probablement bon.»

C'est du Monte-Carlo pur : on estime une valeur inconnue (la qualité d'un coup) par la moyenne de tirages aléatoires (des parties jouées au hasard).

6.2 Le raffinement : ne pas répartir équitablement

Jouer mille parties par coup gaspille : la plupart des coups sont manifestement mauvais après cinquante parties, et continuer à les explorer prive les coups prometteurs de temps de calcul.

MCTS (*Monte Carlo Tree Search*) résout ce dilemme avec une formule qui arbitre entre **exploiter** ce qui marche et **explorer** ce qu'on connaît mal :

$$UCB1(i) = \underbrace{\frac{w_i}{n_i}}_{\text{ce que ça rapporte}} + c \underbrace{\sqrt{\frac{\ln N}{n_i}}}_{\text{curiosité}}$$

où w_i est le total des scores obtenus par ce coup, n_i son nombre de visites, et N le nombre de visites du nœud parent.

- le premier terme est le **taux de victoire** : il pousse vers ce qui a marché;
- le second **grossit quand n_i est petit** : il force à essayer les coups délaissés. Le $\ln N$ au numérateur fait que cette curiosité décroît lentement à mesure qu'on connaît mieux le nœud;

- c règle l'équilibre. $c = \sqrt{2}$ si les scores sont dans $[0, 1]$: c'est la valeur qui découle de la théorie des bandits manchots. Plus grand rend l'IA curieuse, plus petit la rend gourmande.

MCTS est de l'échantillonnage d'importance

C'est exactement l'idée du chapitre 4, appliquée à un arbre : **tirer davantage là où ça compte**. UCB1 est la densité p , et l'arbre se construit lui-même autour des branches qui le méritent.

6.3 D'où vient la valeur d'un nœud

C'est la question qu'il faut se poser, et sa réponse n'a rien d'évident : *comment un système peut-il savoir qu'un nœud vaut mieux qu'un autre, alors que ses parties sont jouées n'importe comment ?*

6.3.1 Deux nombres, et rien d'autre

Un nœud ne stocke aucune connaissance du jeu. Il porte **deux compteurs** :

n_i	visites	combien de fois on est passé par ce nœud
w_i	score cumulé	la somme des résultats des parties passées par lui

La valeur estimée du nœud est leur rapport, w_i/n_i : son **taux de victoire**. C'est tout. Aucune règle experte, aucune évaluation de position — deux entiers mis à jour par la rétropropagation.

6.3.2 Pourquoi des parties jouées au hasard informent quand même

L'objection est légitime : une partie aléatoire n'a rien à voir avec une partie bien jouée. Son résultat ne dit donc *rien* sur la valeur réelle de la position. Trois raisons font que ça marche malgré tout.

1. On ne mesure pas une valeur, on compare des frères. Le taux de victoire absolu ne veut rien dire : une position qui gagne 70 % des parties aléatoires perdrait peut-être toujours contre un joueur parfait. Mais **les deux positions comparées sont évaluées avec la même méthode stupide**. Le biais est commun aux deux, donc il s'annule dans la comparaison. Ce qui reste, c'est l'écart — et l'écart, lui, est informatif.

2. Une position vraiment gagnante a plus de chemins vers la victoire. C'est le fondement statistique. Si vous avez une pièce d'avance, la *proportion* de continuations qui finissent bien pour vous est plus élevée — y compris parmi les continuations médiocres. Les simulations échantillonnent l'espace des suites possibles, et la densité de victoires dans cet espace est une image bruitée, mais corrélée, de la qualité réelle.

3. Le hasard s'efface à mesure que l'arbre grandit. C'est le mécanisme le plus important, et le plus souvent manqué. Au tout début, une simulation est faite à 100 % de coups aléatoires. Mais chaque itération allonge l'arbre, et la **phase de sélection** remplace les premiers coups aléatoires par des coups *choisis* par UCB1.

Itération	Début de la partie	Fin de la partie
1	aléatoire	aléatoire
100	2–3 coups choisis	aléatoire
10 000	15–20 coups choisis	aléatoire

L'arbre transforme progressivement le hasard en jeu

Les statistiques d'un nœud très visité ne reposent presque plus sur du hasard : elles reposent sur des parties dont le début a été *réfléchi* par l'arbre lui-même. La qualité de l'évaluation s'améliore donc toute seule, sans qu'on ait écrit la moindre règle de jeu. C'est ce qui distingue MCTS d'un simple sondage aléatoire : le sondage ne s'améliore jamais, MCTS s'améliore à chaque itération.

6.3.3 Un exemple chiffré, à faire à la main

Un nœud parent a été visité $N = 100$ fois, et porte trois coups. Prenons $c = \sqrt{2} \approx 1,414$, et rappelons que $\ln 100 \approx 4,605$.

Coup	Visites	Taux	Curiosité	UCB1
A	60	0,70	$1,414\sqrt{4,605/60} = 0,392$	1,092
B	30	0,60	$1,414\sqrt{4,605/30} = 0,554$	1,154
C	10	0,70	$1,414\sqrt{4,605/10} = 0,960$	1,659

Lisez ce tableau attentivement, il contient tout le comportement de MCTS :

- **C est choisi**, bien qu'il ait le même taux que A. Pourquoi ? Parce qu'il n'a été vu que dix fois : son 0,70 n'est pas fiable, et la curiosité impose d'aller vérifier ;
- **B passe devant A** alors qu'il gagne moins souvent. Même raison : deux fois moins de visites, donc deux fois plus de doute ;
- **A, le plus visité, est le moins prioritaire** — non parce qu'il est mauvais, mais parce qu'on sait déjà ce qu'il vaut. Continuer à l'explorer n'apprendrait plus rien.

UCB1 ne choisit pas le meilleur coup, il choisit le plus utile à essayer

La formule ne répond pas à « quel coup est le meilleur ? » mais à « quel coup m'apprendra le plus, sachant ce que je sais déjà ? ». Le meilleur coup, on ne le désigne qu'à la **toute fin**, quand le budget est épuisé.

Conséquence importante : **aucun coup n'est jamais définitivement abandonné**. Le terme de curiosité grossit tant qu'un coup n'est pas visité, donc il finira toujours par être re-testé. C'est ce qui empêche MCTS de passer à côté d'un bon coup sur un mauvais démarrage.

6.3.4 On ne compare jamais que des frères

Détail technique, mais il évite une confusion tenace : UCB1 ne compare **que les enfants d'un même parent**. Jamais un nœud profond avec un nœud peu profond, jamais deux nœuds de branches différentes.

C'est ce qui rend le terme $\ln N$ correct : N est le nombre de visites *du parent*, donc la même référence pour tous les frères comparés. Une comparaison globale n'aurait aucun sens, puisque des nœuds situés à des profondeurs différentes ne représentent pas des choix alternatifs.

6.3.5 Est-ce que ça converge vraiment ?

Oui, et c'est démontré. Kocsis et Szepesvári ont établi en 2006 que, avec UCB1 comme règle de sélection, la valeur estimée d'un nœud **converge vers sa valeur minimax** quand le nombre d'itérations tend vers l'infini.

Autrement dit : MCTS n'est pas une heuristique bricolée qui marcherait par chance. C'est un algorithme qui tend vers le jeu *parfait* — simplement, il y tend lentement. Avec un budget fini, on obtient une approximation ; et cette approximation est d'autant meilleure que le budget est grand, sans jamais avoir besoin de savoir évaluer une position.

Le taux de victoire d'un nœud peu visité ne vaut rien

Un coup vu 2 fois avec 2 victoires affiche 100 % : c'est le meilleur taux de tout l'arbre, et cela ne veut strictement rien dire.

C'est précisément pour cette raison que le coup final rendu est le **plus visité**, et non le mieux noté. Le nombre de visites est une statistique stable : il traduit la confiance qu'UCB1 a accordée à ce coup des milliers de fois de suite. Un taux de victoire, lui, peut être un accident.

6.4 Les quatre phases, répétées en boucle

1. **Sélection** — depuis la racine, descendre en choisissant à chaque nœud l'enfant de plus fort UCB1, jusqu'à un nœud dont tous les coups n'ont pas encore été essayés ;
2. **Expansion** — créer un nouvel enfant pour l'un de ces coups jamais tentés ;
3. **Simulation** — depuis ce nouveau nœud, jouer **au hasard** jusqu'à la fin de la partie. Résultat : victoire, défaite ou nul ;
4. **Rétropropagation** — remonter jusqu'à la racine en ajoutant une visite et le score à chaque nœud traversé.

L'erreur n°1, celle qui rend l'IA suicidaire

À la remontée, le score doit être **inversé à chaque changement de joueur**. Une victoire pour vous est une défaite pour l'adversaire.

Si vous remontez le même chiffre partout, votre IA choisit les coups qui font gagner *l'autre* — et elle le fait avec une parfaite assurance statistique. Le symptôme est déroutant : plus vous lui donnez de temps de calcul, plus elle joue mal.

6.5 Le code complet

MCTS – structure de noeud et boucle principale

```
1 struct NkMctsNode {
2     NkMctsNode* parent = nullptr;
3     NkVector<NkMctsNode*> enfants;
4     NkVector<NkCoup> coupsNonEssayes; // vide = noeud pleinement étendu
5     NkCoup coupJoue; // le coup qui mene ICI
6     nk_uint32 visites = 0;
7     float32 score = 0.f; // du point de vue de `joueur`
8     nk_int32 joueur = 0; // qui a joue `coupJoue`
9 };
10
11 NkCoup MeilleurCoup(const NkPlateau& depart, nk_uint32 budget) {
```

```

12 NkMctsNode* racine = CreerNoeud(nullptr, depart);
13
14 for (nk_uint32 iter = 0; iter < budget; ++iter) {
15     NkMctsNode* noeud = racine;
16     NkPlateau etat = depart;           // copie de travail
17
18     // (1) SELECTION : descendre tant que le noeud est etendu.
19     while (noeud->coupsNonEssayes.Empty() && !noeud->enfants.Empty()) {
20         noeud = MeilleurEnfantUCB(noeud, 1.41421356f); // c = sqrt(2)
21         etat.Jouer(noeud->coupJoue);
22     }
23
24     // (2) EXPANSION : essayer un coup encore jamais joue.
25     if (!noeud->coupsNonEssayes.Empty()) {
26         const nk_uint32 k =
27             math::NkRand.NextUInt32(noeud->coupsNonEssayes.Size());
28         const NkCoup coup = noeud->coupsNonEssayes[k];
29         noeud->coupsNonEssayes.Erase(k);
30         etat.Jouer(coup);
31         noeud = AjouterEnfant(noeud, coup, etat.JoueurCourant());
32     }
33
34     // (3) SIMULATION : jouer au hasard jusqu'au bout.
35     NkPlateau sim = etat;
36     NkVector<NkCoup> coups;
37     while (!sim.EstTerminee()) {
38         coups.Clear();
39         sim.CoupsLegaux(coups);
40         if (coups.Empty())
41             break;
42         sim.Jouer(coups[math::NkRand.NextUInt32(coups.Size())]);
43     }
44     const nk_int32 gagnant = sim.Gagnant();
45
46     // (4) RETROPROPAGATION -- Le score DEPEND du joueur de chaque noeud.
47     while (noeud != nullptr) {
48         noeud->visites += 1;
49         if (gagnant == 0)
50             noeud->score += 0.5f;           // partie nulle
51         else
52             noeud->score += (gagnant == noeud->joueur) ? 1.f : 0.f;
53         noeud = noeud->parent;
54     }
55 }
56
57 // Le coup rendu est le PLUS VISITE, pas le mieux note : le nombre de
58 // visites est une statistique bien plus stable qu'un taux de victoire
59 // calcule sur trois parties. Un coup vu deux fois avec deux victoires
60 // affiche 100 % -- et ne veut rien dire.
61 return EnfantLePlusVisite(racine)->coupJoue;
62 }

```

UCB1 – le choix de l'enfant

```

1 NkMctsNode* MeilleurEnfantUCB(NkMctsNode* n, float32 c) {
2     NkMctsNode* meilleur = nullptr;
3     float32 meilleureValeur = -1e30f;
4     const float32 logN = NkMath::Log((float32)n->visites);
5
6     for (nk_size i = 0; i < n->enfants.Size(); ++i) {
7         NkMctsNode* e = n->enfants[i];

```



```

8
9      // Un enfant jamais visite est prioritaire ABSOLU : sa formule
10     // diviserait par zero, et la theorie veut qu'on l'essaie d'abord.
11     if (e->visites == 0)
12         return e;
13
14     const float32 exploitation = e->score / (float32)e->visites;
15     const float32 exploration = c * NkMath::Sqrt(logN / (float32)e->visites);
16     const float32 v = exploitation + exploration;
17     if (v > meilleureValeur) {
18         meilleureValeur = v;
19         meilleur = e;
20     }
21 }
22 return meilleur;
23 }

```

6.6 Ce que MCTS vous donne gratuitement

- **aucune connaissance du jeu n'est requise** : il suffit de savoir énumérer les coups légaux et reconnaître une fin de partie. Le même code marche sur un jeu que vous venez d'inventer;
- **interruptible** : arrêtez-le quand vous voulez, il rend son meilleur coup actuel. Une IA qui doit répondre en 200 ms et une qui a 10 secondes, c'est le même code avec un budget différent;
- **il s'améliore tout seul** avec le temps de calcul, sans réglage;
- **il se parallélise** : plusieurs fils peuvent explorer l'arbre en même temps (avec les précautions du chapitre 7).

Ce que MCTS fait mal

Les positions à **coup unique salvateur** — celles où un seul coup sauve la partie et tous les autres perdent. Le hasard des simulations le trouve rarement, et la moyenne le noie. C'est pourquoi les meilleurs programmes combinent MCTS avec une recherche exhaustive à faible profondeur, ou avec un réseau de neurones qui oriente les simulations (c'est la recette d'AlphaGo : MCTS + un réseau pour remplacer le tirage uniforme).

Le sentir sur un morpion

Implémentez MCTS pour le tic-tac-toe, jeu assez petit pour qu'on connaisse la vérité : bien joué, il est toujours nul.

Faites varier le budget : 100, 1000, 10 000 itérations. À 10 000, votre IA ne doit **jamais** perdre. Puis inversez volontairement le signe de la rétropropagation : elle deviendra suicidaire — et vous reconnaîtrez ce symptôme à vie.

Les pièges

Monte-Carlo a une particularité désagréable : quand il est faux, il ne plante pas. Il rend un nombre plausible. Ce chapitre rassemble les erreurs qui coûtent une journée, et les signes qui les trahissent.

7.1 Le générateur aléatoire

Toute la méthode repose sur la qualité du hasard. Un générateur médiocre introduit des **corrélations** entre échantillons — et des échantillons corrélés valent moins que leur nombre.

N'utilisez jamais `rand()` de la bibliothèque C

Trois défauts, tous réhivitoires ici :

- **période courte** : sur certaines implémentations 2^{31} , ce qui est atteint en quelques secondes de rendu ;
- **bits de poids faible quasi périodiques** : `rand() % 2` peut alterner régulièrement. Tout code qui prend un modulo est contaminé ;
- **état global** : inutilisable en multi-fils sans verrou, et un verrou sur le générateur détruit la parallélisation.

Le dépôt fournit mieux : `math::NkRand` et le XORShift64* de `NKContainers/String/NkStringUtils.cpp`.

Une graine par fil d'exécution, sans exception

Si deux fils partagent la même graine, ils tirent la **même séquence** : vous ne multipliez pas vos échantillons, vous dupliquez le même. Le bruit ne diminue plus, et rien ne l'annonce. La bonne pratique tient en une ligne : semer chaque fil avec un identifiant distinct — le numéro de ligne d'image, l'indice du fil, l'indice du pixel — mélangé par une fonction de hachage. Un générateur par fil, jamais partagé.

Un generateur par fil, seme sans collision

```

1 // Melangeur de bits (finaliseur de MurmurHash3) : deux graines voisines
2 // donnent des sequences totalement decorrelees. Semer avec `indice` brut
3 // produirait au contraire des sequences qui demarrent presque pareil.
4 static inline nk_uint64 NkMelanger(nk_uint64 x) {
5     x ^= x >> 33; x *= 0xFF51AFD7ED558CCDull;
6     x ^= x >> 33; x *= 0xC4CEB9FE1A85EC53ull;
7     x ^= x >> 33;
8     return x;
9 }
10
11 class NkRngLocal {
12 public:
13     explicit NkRngLocal(nk_uint64 indice, nk_uint64 graineGlobale = 0x9E3779B9ull)
14         : mEtat(NkMelanger(indice + graineGlobale)) {

```

```

15         if (mEtat == 0)
16             mEtat = 0x853C49E6748FEA9Bull; // XORShift meurt sur zero
17     }
18
19     nk_uint64 Suivant() { // XORShift64*
20         mEtat ^= mEtat >> 12;
21         mEtat ^= mEtat << 25;
22         mEtat ^= mEtat >> 27;
23         return mEtat * 0x2545F4914F6CDD1Dull;
24     }
25
26     // [0,1) en float32 : on prend les 24 bits de POIDS FORT. Les bits de
27     // poids faible sont les moins bons de tous les generateurs rapides.
28     float32 Flottant() {
29         return (float32)(Suivant() >> 40) * (1.f / 16777216.f);
30     }
31
32     private:
33         nk_uint64 mEtat;
34 };

```

7.2 Le biais silencieux

Rappel du chapitre 2 : un estimateur biaisé est *stable et faux*. Il ne tremble pas, il ment — et l’augmentation du nombre d’échantillons ne fait que resserrer l’estimation *autour de la mauvaise valeur*.

Cause	Symptôme	Correction
oubli de la division par p	résultat faux d’un facteur constant	diviser, toujours
coupure à profondeur fixe	image trop sombre	roulette russe
densité mal normalisée	décalage proportionnel	vérifier que $\int p = 1$
région jamais échantillonnée	contribution absente	garantir $p > 0$ où $f \neq 0$

Le test qui démasque un biais

Faites tourner votre estimateur avec N , puis $100N$ échantillons.

- si l’écart entre les deux résultats **rétrécit d’un facteur 10** (racine de 100), c’est du bruit : tout va bien ;
- si les deux résultats sont **proches l’un de l’autre mais différents de la valeur attendue**, c’est un biais. Aucun temps de calcul ne le corrigera.

Testez toujours sur un cas dont vous connaissez la réponse : une sphère uniformément éclairée, une intégrale calculable à la main, un jeu dont l’issue optimale est connue.

7.3 L’accumulation numérique

Additionner un million de float32 en série perd les petites contributions : quand la somme devient grande, l’ajout d’une petite valeur est absorbé par l’arrondi.

Accumuler en float64, ou par sommation compensée

Le remède le moins cher est d'accumuler en float64 et de ne convertir qu'à la fin. Si la mémoire l'interdit (un buffer d'accumulation d'image en haute définition), employez la sommation de Kahan, qui garde trace de l'erreur d'arrondi et la réinjecte.

Symptôme typique quand on l'ignore : une image qui **cesse de converger** au-delà de quelques milliers d'échantillons par pixel — le bruit ne descend plus, alors que la théorie dit qu'il devrait.

7.4 Les fireflies

Des pixels blancs isolés sur une image sombre. Cause : un échantillon a touché une contribution énorme avec une densité minuscule, donc f/p explose.

- **mauvaise solution** : plafonner brutalement les valeurs (*clamping*). Ça marche visuellement, mais ça **introduit un biais** : on retire de l'énergie qui existait vraiment ;
- **bonne solution** : mieux échantillonner ce qui les cause — tirage direct des lumières, échantillonnage à importances multiples. On s'attaque à la densité, pas au symptôme.

7.5 La parallélisation

Monte-Carlo se parallélise magnifiquement : les échantillons sont indépendants. Deux réserves seulement.

- **le déterminisme**. Une image rendue sur 8 fils ne sera pas identique au bit près à la même rendue sur 4 fils, si l'ordre d'accumulation change. Pour un rendu reproductible — indispensable pour comparer deux versions du moteur — il faut que chaque **échantillon** ait sa graine propre, dérivée de son indice et non de l'ordre d'exécution ;
- **le faux partage** (*false sharing*). Si deux fils accumulent dans des cases voisines du même tableau, ils se disputent la même ligne de cache et le programme ralentit sans raison apparente. Donnez à chaque fil son accumulateur, aligné, et fusionnez à la fin.

7.6 Récapitulatif : la liste à cocher

Avant de croire un résultat Monte-Carlo

1. Est-ce que je divise par la densité de tirage, partout ?
 2. Ma densité est-elle strictement positive partout où la fonction ne l'est pas ?
 3. Ai-je testé sur un cas dont je connais la réponse exacte ?
 4. L'erreur décroît-elle bien en $1/\sqrt{N}$ quand je multiplie les échantillons ?
 5. Chaque fil a-t-il sa propre graine ?
 6. Est-ce que j'accumule dans un type assez large ?
 7. Est-ce que je coupe quelque chose « pour aller plus vite » sans compenser ?
- Sept questions. Elles coûtent dix minutes et font gagner des journées.

7.7 Pour aller plus loin

- **Suites à faible discordance** (Halton, Sobol) : la stratification qui survit à la grande dimension ;
- **échantillonnage à importances multiples** : combiner proprement plusieurs densités, chacune bonne pour une partie du problème ;
- **Metropolis-Hastings et MCMC** : quand la bonne densité est impossible à échantillonner directement, on construit une marche aléatoire qui l'explore. C'est la suite naturelle de ce cours ;
- **estimateurs à variance réduite** : variables de contrôle, variables antithétiques.